Remember that you don’t need to set your URLs in this way if you’re using IIS as a reverse proxy; IIS integration handles this for you. Setting the URLs is necessary only when you’re manually configuring the URL your app is listening on, such as if you’re using NGINX or are exposing Kestrel directly to clients.

WARNING If you are running your ASP.NET Core application without a reverse proxy, you should use *host filtering* for security reasons to ensure that your app only responds to requests for hostnames you expect. For more details, see my “Adding host filtering to Kestrel in ASP.NET Core” blog entry: http://mng.bz/pVXK.

That brings us to the end of this chapter on publishing your app. This last mile of app development—deploying an application to a server where users can access it—is a notoriously thorny problem. Publishing an ASP.NET Core application is easy enough, but the multitude of hosting options available makes providing concise steps for every situation difficult.

Whichever hosting option you choose, there’s one critical topic that you mustn’t overlook: security. In the next chapter you’ll learn about HTTPS, how to use it when testing locally, and why it’s important your production apps all use HTTPS.

Summary

- ASP.NET Core apps are console applications that self-host a web server. In production, you may use a reverse proxy, which handles the initial request and passes it to your app. Reverse proxies can provide additional security, operations, and performance benefits, but they can also add complexity to your deployments.